

# AI Intelligence Graph – Data Ingestion Architecture

**Goal:** Build a scalable, fault-tolerant pipeline for startups, products, research papers, AI jobs, news and entity-resolution records. The same application code can scale from 1,000+ records to 500k+ records by increasing infrastructure.

## 1. High-Level Architecture

Sources ? Async Crawlers ? Raw Storage ? Normalization ? LLM Extraction ? Entity Resolution ? Primary Database ? Vector/Graph Storage ? Google Sheet / API

Source-specific adapters handle RSS, APIs and web pages. Async workers use bounded concurrency, timeouts, retries and caching. Raw responses are retained for traceability and every normalized record keeps its source URL.

## 2. Scale Strategy: 1k to 500k+ Records

| Layer         | Scaling approach                                                                        |
|---------------|-----------------------------------------------------------------------------------------|
| Crawling      | Async workers + semaphore; partition work by source/entity; horizontal worker scaling.  |
| Queue         | Use a durable queue such as SQS/Kafka/RabbitMQ to distribute crawl and enrichment jobs. |
| Storage       | Object storage for raw HTML/JSON; PostgreSQL for canonical records and metadata.        |
| Search        | Vector index for semantic retrieval; graph database for entity relationships.           |
| Deduplication | Stable source IDs + canonical URLs + content hashes + deterministic entity keys.        |
| Observability | Structured logs, metrics, retries, dead-letter queue and per-source health checks.      |

## 3. HTTP 429 and 413 Handling

**429 Rate Limit:** Retry with exponential backoff and random jitter. Example delay = min(base × 2^attempt + jitter, maximum delay). Respect Retry-After when supplied. Use per-source rate limits so one source cannot exhaust worker capacity.

**413 Payload Too Large:** Do not repeatedly resend the same payload. Split long documents into smaller semantic chunks with limited overlap, then process each chunk. If a provider still rejects a chunk, progressively reduce the chunk size.

## 4. Freshness and Distributed Deduplication

News and jobs are filtered using normalized UTC publication timestamps and a 24-hour cutoff. When strict dates are unavailable, a documented content/date heuristic may be used. Distributed workers deduplicate using a deterministic key such as **source + canonical URL** and a content hash. Database unique constraints make writes idempotent even when multiple workers process the same item.

## 5. LLM Fallback Chain

The extraction layer uses a provider fallback chain such as **Gemini Flash ? Groq model ? DeepSeek**. Each provider is isolated behind an adapter. Validation uses Pydantic schemas. A failed provider does not stop the pipeline; the next provider is attempted. Secrets are stored only in environment variables.

## 6. Async Crawler and Difficult Sources

The crawler uses Python asyncio/aiohttp with bounded concurrency, connection timeouts, retries and response caching. Playwright Async is available for JavaScript-rendered pages. For Cloudflare, Datadome or CAPTCHA-protected sources, the system does **not** bypass security controls. It uses permitted APIs, official feeds, compliant browser rendering, rate limiting or a source-specific adapter instead.

## 7. Storage Design

**Primary DB:** PostgreSQL for canonical entities, source metadata, timestamps, deduplication keys and audit logs. **Vector storage:** pgvector or a vector database for semantic search over descriptions, papers and news. **Graph storage:** Neo4j or PostgreSQL graph-compatible structures for Startup–Founder–Product–Paper–Job–News relationships. Raw source documents remain in object storage.

## 8. Fault Tolerance and Traceability

Failures are isolated per source and per record. Retries use bounded exponential backoff; permanent failures go to a dead-letter queue for later replay. Checkpoints allow workers to resume without restarting the full crawl. Every record stores source URL, retrieval time, normalized date, content hash and processing status, making the dataset auditable and traceable to its original source.